

We would like to develop a numerical method to approximate the coefficients of basis functions that minimize a functional, i.e. Ritz method. We are interested in functionals of the form

$$J[y] = \int_a^b F(x, y, y') dx$$

which satisfy $y(a)=A$, $y(b)=B$. These constraints are satisfied by user defined basis functions of the form

$$\bar{y} = \phi_0 + \sum_{i=1}^n a_i \phi_i$$

Stated again, the goal is to find a_i ($i=1, \dots, n$) that minimize the functional.

Method Brainstorm

The solution is approximated by

$$\bar{y} = \phi_0 + \sum_{i=1}^n a_i \phi_i$$

The derivative of $\bar{y}$ with respect to x is then

$$\bar{y}' = \phi_0' + \sum_{i=1}^n a_i \phi_i'$$

Substituting this approximation into J i.e. $J[y] = J[\bar{y}]$ leads to

$$\phi(a_1, \dots, a_n) = \int_a^b F(x, \phi_0 + \sum_{i=1}^n a_i \phi_i, \phi_0' + \sum_{i=1}^n a_i \phi_i') dx$$

We're looking for an efficient way to evaluate this integral that works with either BFGS or LBFGS, cause that would be the most simple. Ideally this will also work with autodiff cause I don't want to fool with tuning step sizes and autodiff will be more accurate.

We'll use quadrature to evaluate the integral. Since we are optimizing over the basis function coefficients, the basis functions ~~will not change values~~ at the nodes will be constant at the quadrature nodes, and can be precomputed.

Ritz method function skeleton code

Ritz(a, b, ϕ_0 , [$\phi_1 \dots \phi_n$], num.quad.nodes)

- Calculate quadrature nodes (quass nodes for us) + weights
- Change of interval of nodes
- Calculate basis function (and derivative) values @ nodes
- Create objective function
- Find minimum of of objective functions
- Report results

end

Random Notes + Brainstorm + Brain Dump

I suspect that since the approximations are sums and products, we can elegantly precompute these values. Actually it's exactly a linear system. Let's consider a 3 term basis with 2 nodes before we generalize.

~~It'll try with~~

$$\begin{bmatrix} \bar{y}(x_1) \\ \bar{y}(x_2) \end{bmatrix} = \underbrace{\begin{bmatrix} \phi_1(x_1) & \phi_2(x_1) & \phi_3(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \phi_3(x_2) \end{bmatrix}}_{\phi} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \cancel{\begin{bmatrix} \phi_0(x_1) \\ \phi_0(x_2) \end{bmatrix}}_{\phi_0}$$

The same pattern can be use for the derivative

$$\begin{bmatrix} \bar{y}'(x_1) \\ \bar{y}'(x_2) \end{bmatrix} = \underbrace{\begin{bmatrix} \phi_1'(x_1) & \phi_2'(x_1) & \phi_3'(x_1) \\ \phi_1'(x_2) & \phi_2'(x_2) & \phi_3'(x_2) \end{bmatrix}}_{\phi'} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \underbrace{\begin{bmatrix} \phi_0'(x_1) \\ \phi_0'(x_2) \end{bmatrix}}_{\phi'_0}$$

So we just need to make these matrices, which can probably be done elegantly with list comprehension.

The quadrature is then just a dot product of the integrand and quadrature weights, multiplied by the change in interval coefficient.

~~★~~ Don't forget change of interval coefficient